

Explicit Criteria & False Positives

Tell Claude exactly what 'good' looks like.

IN THIS LECTURE,

1 Why criteria matter

2 Vague vs specific (+ anatomy)

3 Severity levels

4 The false-positive problem

5 Why noise erodes trust

6 Tuning noisy categories

Why Explicit Criteria Matter

Claude judges against the yardstick you give it.



Vague ask

- ✗ 'Is this code good?'
- ✗ Claude invents a standard — it drifts

VS



Explicit criteria

- ✓ A clear checklist to judge against
- ✓ The same standard every time

 Vague ask → Claude invents a standard that drifts run to run. Explicit criteria → the same standard every time.

Vague vs Specific

'Check for security issues' is a wish; a specific rule is testable.



Vague (a wish)

- ✗ 'Check for security problems.'
- ✗ Claude has to guess what you meant



Specific (testable)

- ✓ 'Flag any SQL query built with string concatenation.'
- ✓ Specific · observable · testable



A criterion Claude can't test objectively will be applied inconsistently. Write rules you could check by hand.

Severity Levels

Not every issue is a five-alarm fire.



Critical Blocks the release — e.g. a hard-coded password



Major Fix soon — e.g. missing error handling



Minor Nice to fix — e.g. an inconsistent name



Without severity, a typo and a security hole look equally urgent — and the important issue gets buried.

The False-Positive Problem

Flagging things that aren't actually problems.

A false positive = Claude flags something that is fine

- ▶ Over-broad criteria → many false alarms
- ▶ Like a smoke alarm going off at toast
- ▶ The opposite is a false negative — missing a real issue



Both hurt — but false positives are the trust-killer, as the next slide shows.

Why Noise Erodes Trust

A reviewer who cries wolf gets ignored.



Too many false alarms → people stop reading the output

- ▶ 18 nonsense flags bury the 2 real ones
- ▶ Users click 'dismiss, dismiss, dismiss'
- ▶ Then they scroll past the genuine critical flag



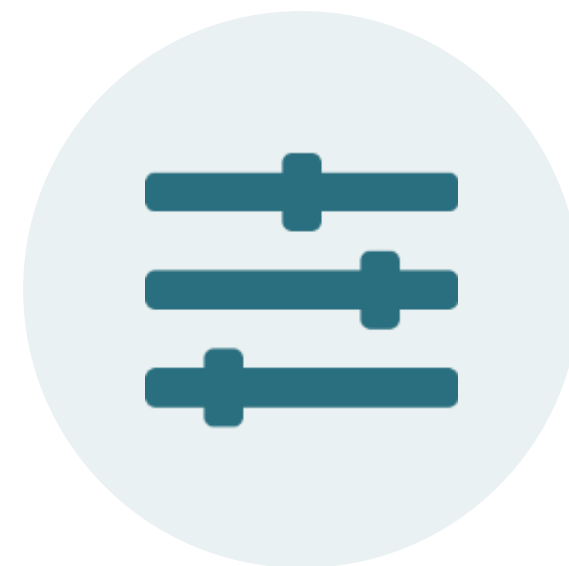
A noisy reviewer is worse than none — users tune it out entirely, so even its correct flags stop working.

Tuning: Disable the Noisy Categories

Turn off what you can't make precise.

Narrow it, or switch it off

- ▶ If a category (e.g. 'style nitpicks') is mostly false positives...
- ▶ ...tighten its rule, or disable that category
- ▶ Fewer, high-precision flags beat many noisy ones



Over-flagging problem → tighten or disable the noisy category — don't just add 'be more careful' to the prompt.

Worked Example: Two Review Prompts

Same code, two very different reviewers.



'Review this code'

✗ 15 flags

✗ Mostly style noise — the team ignores it



Criteria + severity + tuned

✓ 3 flags: 1 critical, 2 major

✓ All real — the team acts on it



The tuned reviewer finds FEWER things — and that's exactly why the team acts on it.

Key Takeaways

Explicit criteria, in three lines.

1

Specific & testable

Give Claude specific, testable criteria
— not vague wishes.

2

Add severity

Severity levels prioritize findings so
the important ones surface.

3

Control false positives

Tune or disable noisy categories to
keep the reviewer trusted.



End of Lecture 4.1 — Explicit Criteria & False Positives

Few-Shot Prompting

Don't just tell Claude what you want — show it.

IN THIS LECTURE,

1 Zero → one → few-shot

2 Why examples beat description

3 Pinning meaning & format

4 Generalization

5 Reducing hallucination

6 Choosing good examples

7 How many & ordering

Zero → One → Few-Shot

The number of examples you show has a name.

0

Zero-shot

No examples — just instructions



1

One-shot

One example



1 2
3 4

Few-shot

Several examples — the clearest pattern



'Shot' just means 'example.' Few-shot = 'here are a few examples, now do the same.'

Why Examples Beat Description

One good example can replace a paragraph of rules.



Description only

- ✗ A wall of rules for tone & format
- ✗ Hard to write, easy to misread

VS



One example

- ✓ Shows tone, format, edge-handling at once
- ✓ Show, don't tell



An example carries details you'd struggle to write as rules.

Pinning Meaning and Format

Examples kill ambiguity and lock the output shape.

Reduce ambiguity

Show what you actually mean by a fuzzy label like 'urgent'



Format consistency

Show the exact shape to return — Claude copies it every time



A classifier that's inconsistent about a fuzzy label, or messy about format, is usually fixed by adding examples.

Generalization

Claude copies the pattern, not the literal examples.

Good examples teach the underlying rule

- ▶ Claude handles NEW inputs it hasn't seen
- ▶ Show 3 messy→clean addresses; it cleans a 4th
- ▶ Training by demonstration



💡 The goal is the pattern behind the examples — not memorizing those exact cases.

Reducing Hallucination

Examples show the safe way to handle 'I don't know.'



Include an example where information is missing

- ▶ Show a case where the right answer is 'not found' / null
- ▶ Claude learns to NOT invent data
- ▶ One honest example teaches honesty



If every example is a 'full' case, Claude assumes it must always produce a value — and fabricates. Show a missing-data case.

Choosing Good Examples

Diverse, correct, and covering the edge cases.

Correct

A wrong example teaches the wrong thing



Diverse

Not three near-identical easy cases



Edge cases

Ambiguous input, missing field, odd format



Three near-identical easy examples teach almost nothing. Diversity + edge cases is what makes few-shot work.

How Many, and In What Order

Enough to show the range — kept consistent and clean.

A few, uniform, and placed before the input

- ▶ Often 2–5 examples — stop when accuracy plateaus
- ▶ Keep every example's format identical
- ▶ Put the examples BEFORE the real input



Inconsistent formatting across your examples confuses the pattern — keep them uniform.

Key Takeaways

Few-shot prompting, in three lines.

1

Show, don't tell

Examples pin meaning and format —
and generalize to new inputs.

2

Pick good examples

Correct, diverse, cover edge cases —
incl. a missing-data → null case.

3

Few, uniform, first

Use a few, keep the format identical,
place them before the input.



End of Lecture 4.2 — Few-Shot Prompting

Structured Output with Tool Use

Get back clean JSON your code can trust — every time.

IN THIS LECTURE,

- 1 Why free text is unreliable
- 2 tool_use forces structure
- 3 The JSON Schema
- 4 Field types (required/optional/nullable)
- 5 Enums + 'other'
- 6 tool_choice
- 7 What it does NOT fix

Why Free Text Is Unreliable

Your code can't depend on a paragraph.

'Usually valid' isn't good enough for automation

- ▶ Claude may add 'Here's the data:' or markdown fences
- ▶ ...or quietly rename a field
- ▶ One stray sentence and the parser breaks



? For a chat reply that's fine; for a pipeline feeding a database, it fails. We need a guarantee, not a hope.

tool_use Forces Structure

Claude has no dedicated JSON mode — it uses tools.



Define a tool; Claude's answer IS the tool's input

- ▶ Give a tool an `input_schema` describing your fields
- ▶ Claude 'calls' the tool with structured data
- ▶ You get a clean `tool_use` block, not prose



Anthropic's structured-output pattern = forced tool use. You define the shape as a tool schema; Claude fills it in.

The JSON Schema

The schema is your blueprint for the answer.

input_schema — every field, typed

extract_claim · input_schema

```
{  
  "claim_id": "string",  
  "amount":   "number",  
  "date":     "string"  
}
```

Names every field, its type, and which are required.



A malformed schema is rejected up front (a 400 error) before Claude even runs — validate your schema first.

Field Types: Required vs Nullable

Make a field required and Claude will invent it when it's missing.

The root-cause fix for fabricated fields

Required → fabricates

```
"phone": {  
  "type": "string"  
}  
// missing? Claude  
// invents a number
```

Nullable → honest

```
"phone": {  
  "type":  
    ["string", "null"]  
}  
// missing? → null
```

⚠️ Root-cause fix: make fields the document **MIGHT NOT** have optional or nullable — don't just say 'don't guess.' Mark only always-present fields required.

Enums and the 'other' Escape Hatch

Constrain a field to a fixed list — with a safety valve.

Enum for consistency · 'other' + detail for honesty

claim type · enum + escape

```
"type": {  
  "enum": ["auto", "home",  
           "health", "other"]  
},  
"detail": "string"  
// used when type = other
```



Enum with no escape → Claude jams odd inputs into the closest wrong option. 'other' + detail keeps it honest.

tool_choice: Guaranteeing the Call

Make sure Claude actually uses the tool.

auto

May reply in text — NO guarantee



any

Forces some tool — use this for structured output



tool

Forces this specific named tool



↪ We taught the four modes in 2.3 — here we apply them. Left on 'auto', Claude can skip the tool and hand you prose.

What Structured Output Does NOT Fix

Valid JSON can still be wrong.



Syntax — SOLVED

- ✓ Malformed JSON — guaranteed gone
- ✓ tool_use always returns well-formed data



Semantic — NOT solved

- ✗ Numbers that don't add up
- ✗ Wrong field · fabricated value



Structured output guarantees the SHAPE, never the TRUTH. Checking the meaning is the next lecture.

Key Takeaways

Structured output, in three lines.

1

Force the shape

tool_use + a JSON Schema; use tool_choice (any/named) to guarantee the call.

2

Design against fabrication

Optional/nullable for uncertain fields; enums + 'other'/detail escape.

3

Syntax, not meaning

It kills syntax errors — it never guarantees the values are correct.



End of Lecture 4.3 — Structured Output with Tool Use

Validation & Retry Loops

Catch the wrong-but-valid answers — and fix them.

IN THIS LECTURE,

1 Syntax vs semantic errors

2 The validation step

3 Retry with feedback

4 Retry limits

5 The self-correction loop

6 A worked loop

Syntax vs Semantic Errors

Two ways an answer can be wrong.



Syntax error

- ✓ Malformed JSON (broken shape)
- ✓ Already solved by tool_use

VS



Semantic error

- ✗ Valid shape, wrong content
- ✗ Bad sums · wrong field · fabricated value



The distinction the exam leans on: tool_use kills syntax errors; only validation catches semantic ones.

The Validation Step

After extraction, verify the content with your own code.

Check the values against your rules and the source

- ▶ Do the line items sum to the total?
- ▶ Is the date real and in range?
- ▶ Does every value actually appear in the document?



💡 **Common semantic checks: totals add up · dates/ranges are sane · no value appears that isn't in the source (fabrication).**

Retry With Feedback

Don't just fail — tell Claude what was wrong.



Send back: document + bad extraction + the specific error

- ▶ Ask Claude to correct it
- ▶ 'Line items sum to ₹4,800, but total says ₹5,000 — please fix'
- ▶ A failure becomes a guided correction



The feedback must be specific. 'It's wrong' barely helps; 'the total doesn't match the line items' fixes it fast.

Retry Limits

Loop a few times — then stop.

Cap retries (commonly 2–3), then escalate

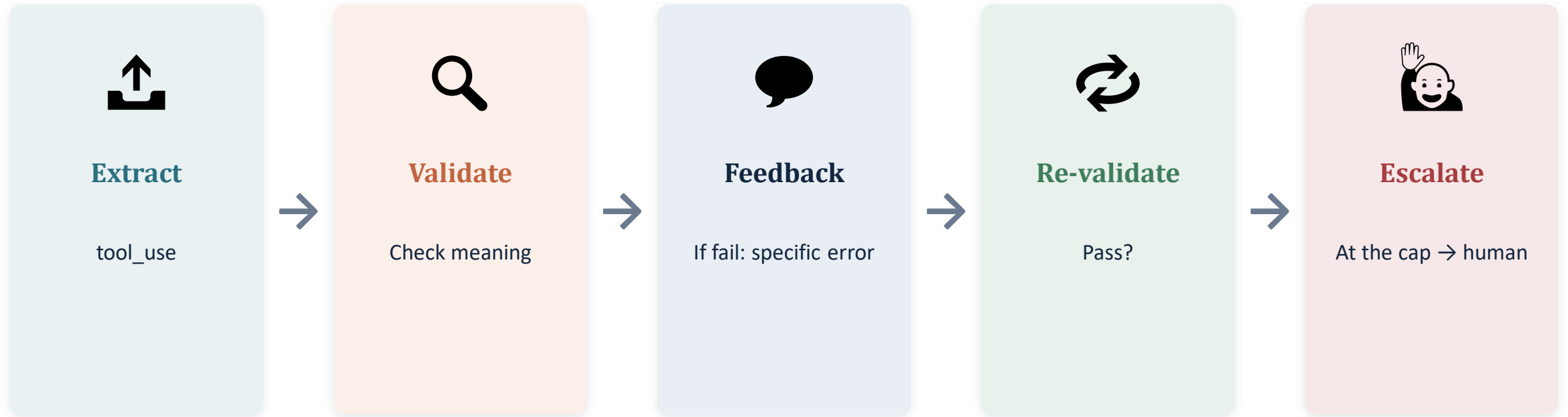
- ▶ Each retry with feedback removes most remaining errors
- ▶ After the cap → flag for a human / mark low-confidence
- ▶ Don't loop forever on an input Claude can't fix



No cap = an infinite loop burning time and money on an input Claude may simply be unable to fix.

The Self-Correction Loop

Extract → validate → feedback → re-validate → stop.



This loop is self-correcting — Claude fixes its own mistakes when told precisely what they are.

Worked Example: The Claim That Didn't Add Up

One bad total, fixed in one retry.

Valid JSON both times — only validation caught it

Attempt 1 · fails validation

```
items: [2000, 2800]
total: 5000      ✕

validator:
"sum is 4800"
```

Attempt 2 · after feedback

```
items: [2000, 2800]
total: 4800      ✓

validator:
"passes"
```



The JSON was valid both times — only the validation step caught the wrong number.

Key Takeaways

Validation & retry, in three lines.

1

Validate the meaning

tool_use kills syntax errors; validation catches semantic ones (sums, wrong field, fabrication).

2

Retry with feedback

On failure, send the specific error so Claude self-corrects.

3

Cap, then escalate

Cap retries at 2–3, then hand off to a human — never loop forever.



End of Lecture 4.4 — Validation & Retry Loops

Batch Processing

Half the price — if you can wait.

IN THIS LECTURE,

1 What batch processing is

2 The cost/latency trade-off (& when to use it)

3 custom_id

4 The submit → poll → retrieve workflow

5 A worked example

What Batch Processing Is

Submit a big pile of jobs; collect the results later.

Many requests in one submission, processed in the background

- ▶ The Message Batches API runs them asynchronously
- ▶ You come back for the results
- ▶ Same model, same quality — just not instant



? Like dropping films at a photo lab overnight instead of waiting at a one-hour booth — same work, cheaper, not instant.

The Trade-off — and When to Use It

50% cheaper, up to 24 hours — no real-time.

50%

off input AND output tokens · plus a separate rate-limit pool



- ▶ Results within 24h (often minutes–hours)
- ▶ No streaming — not for anything a user waits on
- ▶ ✓ bulk extraction, evals, moderation ✕ chat, live UI



Decision rule: 'Is it OK for the user to get this within 24 hours?' Yes → batch. No → the normal synchronous API.

custom_id: Matching Results to Inputs

Label every request so you know which answer is which.



Results come back out of order

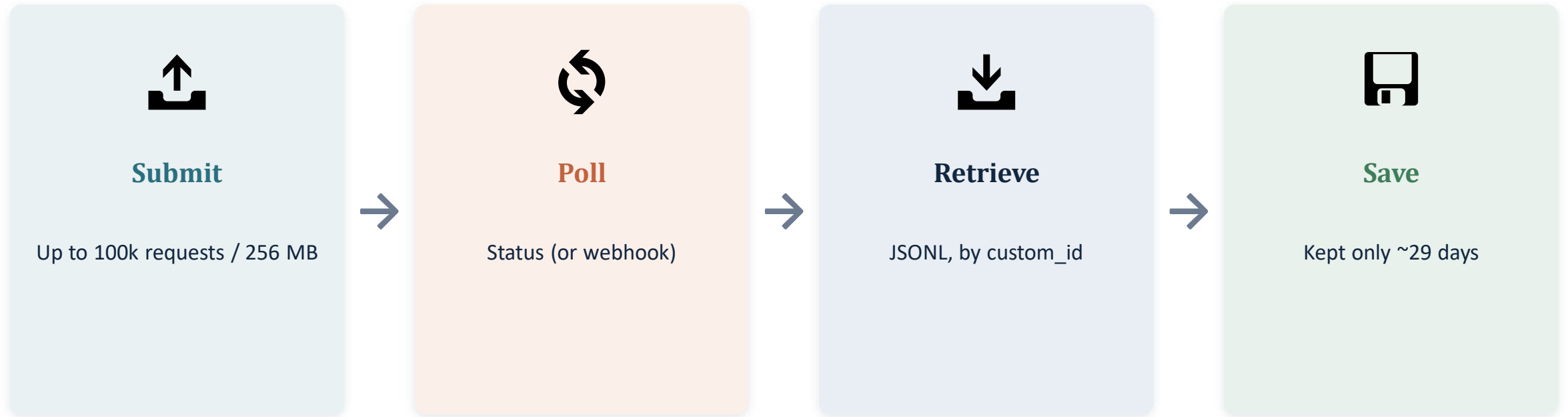
- ▶ Give each request a custom_id (e.g. the claim number)
- ▶ Match each result back by its id
- ▶ Use your own record id — not a throwaway



Without a meaningful custom_id, you can't tell which result belongs to which document.

The Workflow

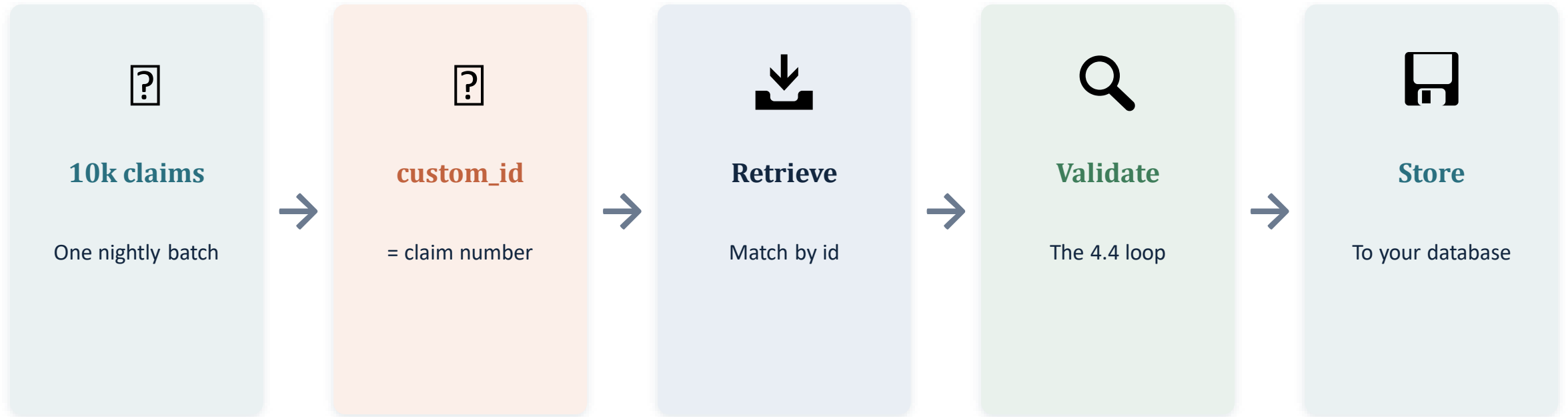
Submit → poll → retrieve → save.



📌 Results are retained ~29 days — persist them immediately. Huge jobs? Split into multiple sequential batches.

Worked Example: A Month of Claims

10,000 claims, half the cost, overnight.



💡 Batch pairs perfectly with earlier lectures: same schema (4.3), and each result still runs through validation (4.4).

Key Takeaways

Batch processing, in three lines.

1

Async & 50% cheaper

Results within 24h — for non-urgent, high-volume work.

2

Tag with custom_id

Results return shuffled — match them by a meaningful id.

3

Submit→poll→retrieve→save

Persist results fast (~29-day retention); never use it when a user waits.



End of Lecture 4.5 — Batch Processing

Multi-Instance & Multi-Pass Review

Two sets of fresh eyes beat one tired glance.

IN THIS LECTURE,

1 Why one pass isn't enough

2 The independent reviewer

3 Per-file + cross-file passes

4 Confidence-annotated passes

5 Aggregating the passes

Why One Pass Isn't Enough

A single reviewer, in one sweep, misses things.

Limited attention, plus self-review bias

- ▶ One sweep over many files → focus spreads thin
- ▶ The author reviewing its own work is biased to approve
- ▶ We fix both — with independence and multiple passes



Two problems: limited attention across a big task, and bias when the reviewer also wrote the work.

The Independent Reviewer

A fresh instance that didn't write the work.



A separate Claude instance, with its own context

- ▶ It never saw the author's reasoning or excuses
- ▶ So it judges the work purely on its merits
- ▶ One instance builds; a different one reviews

🔗 This is the isolation idea from 3.6 (self-review) and the subagents of 1.2–1.3 — a clean context gives an unbiased opinion.

Per-File and Cross-File Passes

Look closely at each part — then at how they fit.



Per-file pass

- ✓ Review each file on its own, deeply
- ✓ Catches local bugs & style



Cross-file pass

- ✓ How the files work together
- ✓ Integration, mismatched interfaces



A per-file-only review misses integration bugs; a cross-file-only review misses local ones. Multi-pass = both.

Confidence-Annotated Passes

Each finding says how sure it is.

Attach a confidence to every finding

- High / medium / low on each flag
- High-confidence → act on it
- Low-confidence → route to a human



Confidence turns a flat list into a routing signal — sure things auto-apply, unsure things get human eyes.

Aggregating the Passes

Combine the passes into one clean verdict.



Merge — don't just concatenate

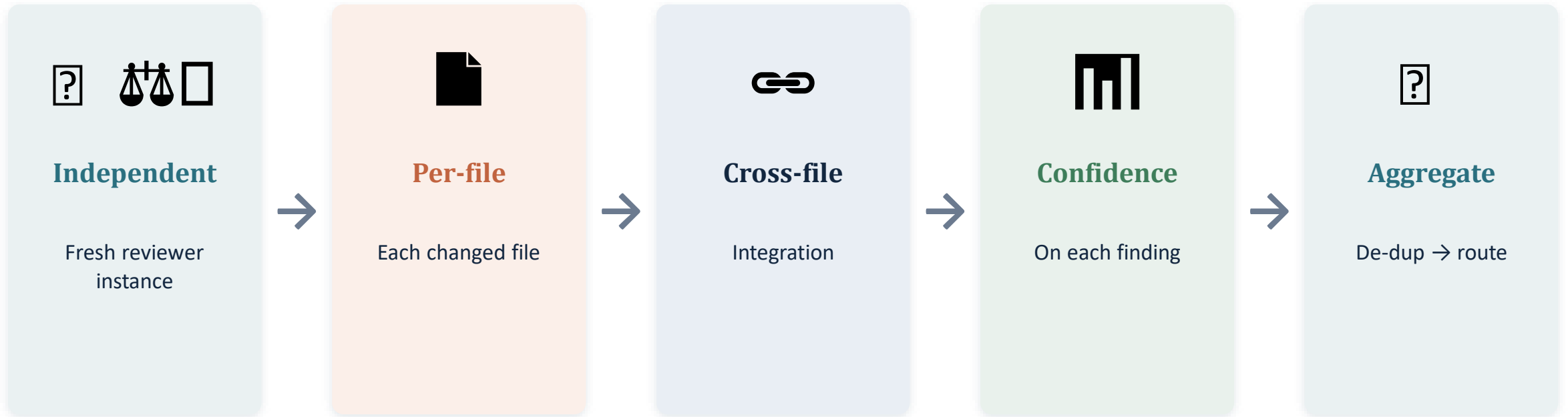
- ▶ De-duplicate overlapping findings
- ▶ Keep the highest severity when they overlap
- ▶ Group by confidence for routing



Two passes flagging the same issue should appear ONCE, at its true severity — or you're back to the trust problem from 4.1.

Worked Example: Reviewing a Feature Branch

The whole architecture on one change.



🔗 This is the architecture 1.6 and 3.6 pointed to — decomposition (per-file/cross-file) meets independent, confidence-scored review.

Key Takeaways

Multi-pass review, in three lines — and Domain 4 wrapped.

1

Independent reviewer

A separate instance kills self-review bias.

2

Two passes + confidence

Per-file AND cross-file passes; annotate each finding with confidence.

3

Aggregate & route

De-dup, keep top severity, and send low-confidence findings to humans.



End of Lecture 4.6 — Multi-Instance & Multi-Pass Review